

TalentSignal Revenue Engine

Revision & Q&A study guide · Releases 0–3 · for Megan

How to use this. Read it once, then cover the answers and quiz yourself on the questions. Anything you can't say out loud in plain words is a soft spot. The goal isn't memorizing — it's being able to explain each idea with its analogy, in your own words, with no notes.

Part 1 — the whole project in plain words

What it is. TalentSignal helps a staffing agency — a matchmaker between companies that need workers and people who need jobs — spot which companies are about to need staff *before* they announce it, so a salesperson can call first, ahead of rival agencies.

The one rule that governs everything. The AI drafts and suggests; a human reviews and releases anything that would leave the platform. Nothing goes out on its own.

Analogy: a newsroom — reporters (the AI) write the story, but an editor (a human) decides what actually gets published.

Part 2 — what each release does

R0 — Walking skeleton (S-01 to S-03)

The frame of the house before the rooms. One command starts all three layers (React front end, Node API, PostgreSQL database) and proves they talk. Adds secure login, and pushes one mocked signal all the way to a scored opportunity on screen.

Analogy: Building a house: pour the foundation and frame before painting a wall.

R1 — Core intelligence (S-04 to S-07)

The clever part — but transparent. Ranks companies by hidden demand (with a confidence score and reasons), ranks candidates for a job by skill overlap (showing which skills matched), models clients and candidates with personal data tagged for privacy, and scores each opportunity consistently 0–1 with a factor breakdown.

Analogy: A doctor weighing symptoms — and always able to explain why they're worried.

R2 — Pipeline & human-in-the-loop (S-08 to S-11)

The heart of the trust story. A live sales pipeline with an append-only audit trail; AI-drafted outreach packages a human must release; warm-relationship intros a human confirms; and recommendations with a thumbs-up/down feedback loop.

Analogy: An assistant writes the letter and leaves it on your desk — only you sign and mail it.

R3 — Analytics & compliance (started: S-12)

The manager's dashboard: placements per month, time-to-hire, and demand score — computed from real data and charted. It shows only aggregates, never any individual's personal data. (Forecasting, access control, and compliance are still ahead.)

Analogy: A crowd photo from above — count the crowd, but you can't identify any face.

Part 3 — concepts you must be able to explain

Confidence score

A number from 0 to 1 saying how likely a company secretly needs staff. It adds up clues — a reposted job, a role open a long time, no salary listed — each weighted by how much it matters. Always shown WITH its reasons, never a bare number.

Analogy: A grade made of parts: homework 20%, midterm 30%, final 50%. The weights are the judgment.

The append-only audit log

Every important change writes a permanent row: who, when, what. The DATABASE itself rejects any edit or delete — even an administrator can't rewrite history. Enforced by a database rule (a trigger), not by polite code.

Analogy: A ship's logbook in ink — add a new line, never tear out a page.

Human-in-the-loop / the release gate

The AI composes outreach but stores it as a DRAFT, delivered nowhere. Only an explicit human click marks it released, recording who and when. Nothing is actually sent — the release flag IS the boundary.

Analogy: Writing the letter vs. signing and mailing it. The AI writes; you send.

Aggregate-only analytics

The dashboard shows counts, averages, and rates — never one person's name or contact details. Personal data never reaches the reporting layer.

Analogy: A crowd photo from above: you see the crowd size, not the individual faces.

Matchmaking (skill overlap)

Ranks candidates for a job by how much their skills overlap the job's requirements, and shows which skills matched. Experience adds a bonus, but only for someone who already has a real skill match.

Analogy: Matching puzzle pieces — the more edges line up, the better the fit.

Mocked behind a seam

The outside data (the job board) is fake for now, but hidden behind a clean boundary so a real feed swaps in by changing one line — nothing else has to change.

Analogy: A real engine on a test bench with simulated fuel — the engine genuinely runs.

Part 4 — questions you might be asked

Cover the answers and try each in your own words first.

Q — Did you build this, or did the AI?

The AI wrote most of the code. My job was to specify what to build, review it, and decide whether it was right — and I rejected plenty. Several times it reported a story done when it wasn't, and catching that is the actual job.

Q — Why does the interface look so plain?

Deliberately. The priority was getting the function and the trust guarantees right before polish. The dashboard is the first properly visual screen; styling comes later.

Q — How do you know the audit log can't be tampered with?

A database trigger rejects any UPDATE or DELETE on audit rows — even from an administrator, because a trigger sits outside the permission system. I proved it with a test that fires an edit and a delete at the log and confirms the database refuses both.

Q — Where did the scoring weights come from?

Me. They're a documented proposal, flagged as pending the client's approval — not client-approved numbers. The specification says these are a business decision I shouldn't invent silently, so I proposed reasonable values transparently and put them in one place that's a one-line change if the client wants different.

Q — Is it secure?

Authentication yes — hashed passwords, expiring tokens, rate limiting, and login reveals nothing about who has an account. Authorization not yet: roles are stored but not enforced on every route — that's the next release. And nothing's deployed, so it hasn't faced the public internet.

Q — Can it handle a thousand users at once?

I haven't measured it, so I won't claim a number. Load testing is a dedicated later story. I'd rather tell you that honestly than guess.

Q — What's real and what's mocked?

The stack is real — real database, real API, real browser, real tokens. Only the outside job-board feed is mocked, behind a clean seam, so a real one swaps in with one line.

Q — Why is the AI not allowed to send anything itself?

That's the core rule — trust before intelligence. Anything that would leave the platform or write to a client's system needs an explicit human release, and it's enforced in code, not by convention. It protects the agency's relationships and the candidates' trust.

Q — What was the hardest bug you caught?

A rate limiter that was accidentally scoped to the entire app instead of just login — a couple of page reloads would have frozen everything with errors, live in the demo. It passed every existing test because the tests only checked the intended behavior, not the side effect. I found it by running the real app end to end.

Q — What would you do differently?

Install Docker before starting — it cost me time on day one. And I'd reconnect with the client sooner on deployment, since that's the one thing I can't build alone.

The three things to remember walking in.

- Lead with the trust story, not the tech — “the AI drafts, a human releases” is the whole pitch.
- Volunteer your limits before you're asked — it makes everything else believable.
- Every analogy in this guide is a lifeline — if you blank on the concept, reach for the picture.